How Statsig Streams 1 Trillion Events A Day

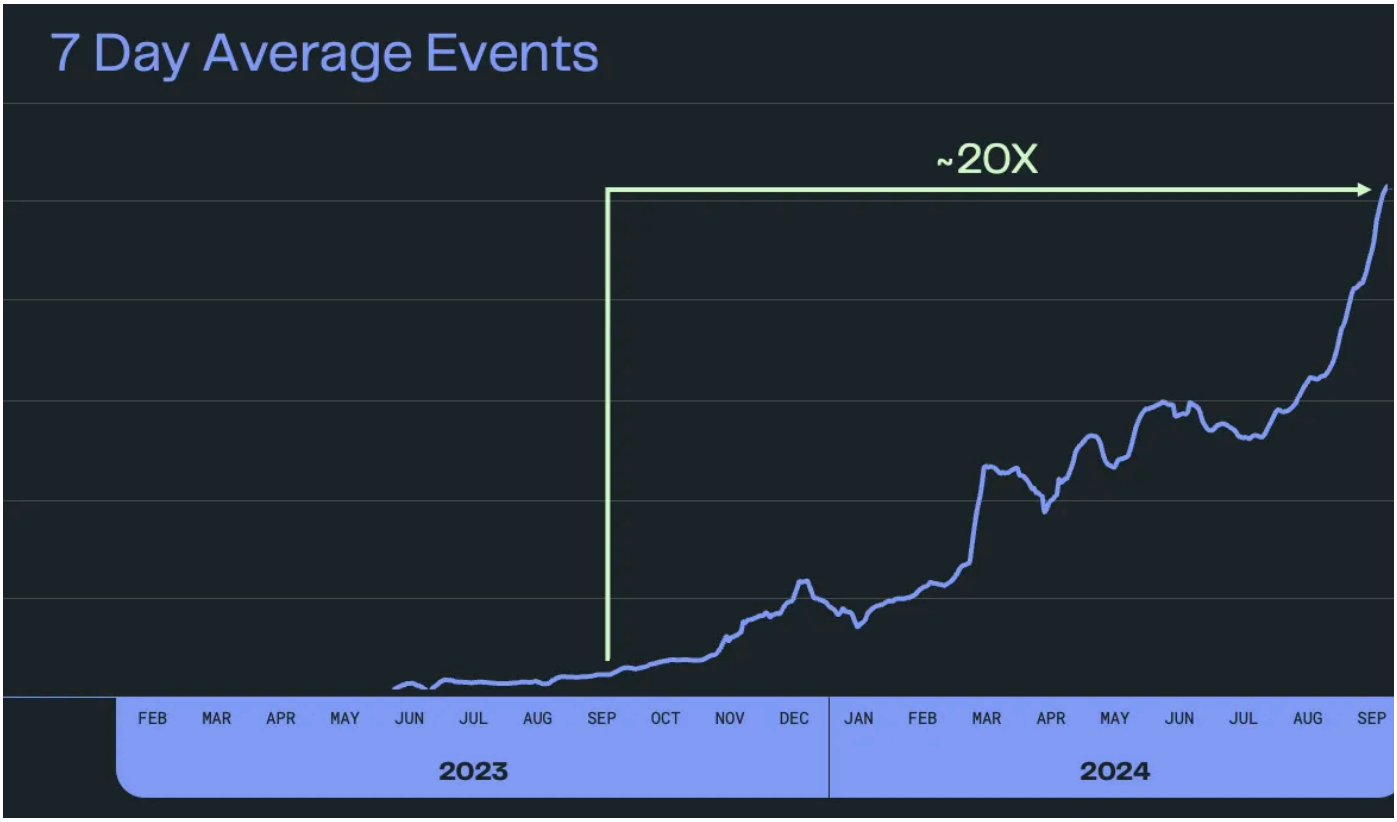
BYTEBYTEGO
DEC 17, 2024

The newsletter is a collaboration between ByteByteGo and members of the Statsig engineering team (Pablo Beltran and Brent Echols).

Statsig is a modern feature management, experimentation, and analytics platform t enables teams to improve their product velocity.

Over the past year, Statsig has seen a staggering 20X growth in its event volume. Th growth has been driven by high-profile customers such as OpenAI, Atlassian, Flipk and Figma, who rely on Statsig's platform for experimentation and analytics.

Statsig currently processes over a trillion events daily, which is a remarkable achievement for any organization, particularly one of their size as a startup.

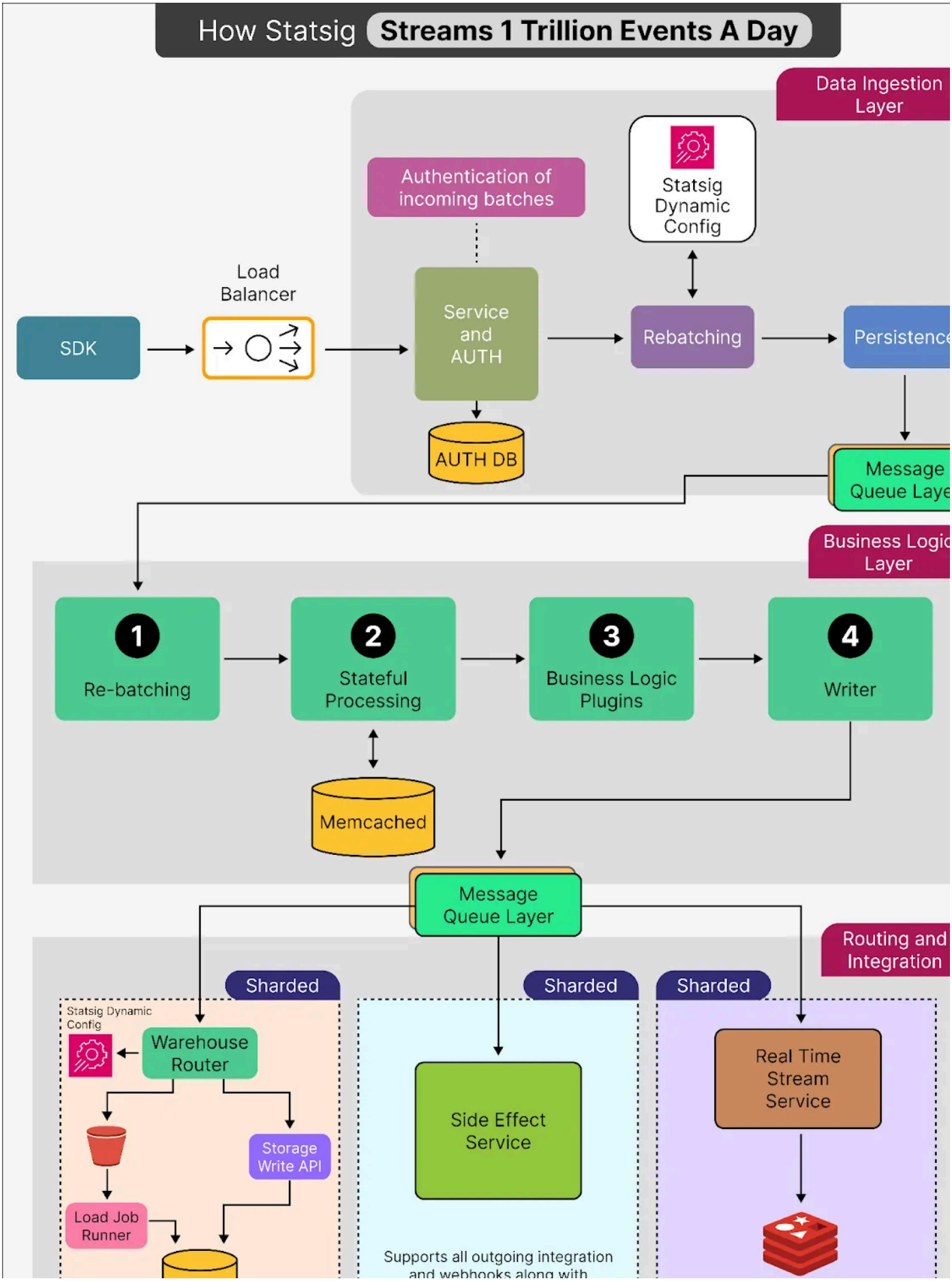


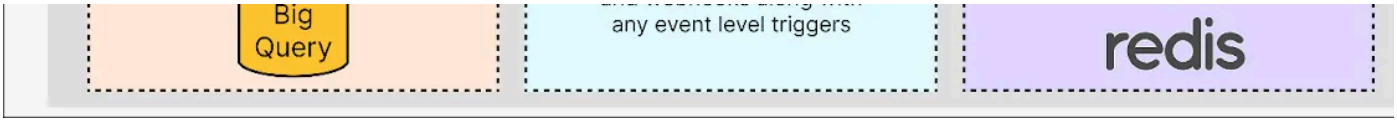
Source: Statsig Technical Blog

However, this rapid growth comes with immense challenges.

Statsig must not only scale its infrastructure to handle the sheer volume of data but also ensure that its systems remain reliable and maintain high uptime. It also needs keep the costs in check to stay competitive.

In this post, we'll look at Statsig's streaming architecture, which helps it handle the event volume. We'll also look at the cost-efficiency steps taken by Statsig's team.


How Statsig Streams 1 Trillion Events A Day
Data Ingestion Layer
Authentication of incoming batches
Statsig Dynamic Config
Load Balancer
SDK
Service and AUTH
Rebatching
Persistence
AUTH DB
Message Queue Laye
Business Logic Layer
1
Re-batching
2
Stateful Processing
3
Business Logic Plugins
4
Writer
Memcached
Message Queue Layer
Routing and Integration
Sharded
Sharded
Sharded
Statsig Dynamic Config
Warehouse Router
Storage Write API
Load Job Runner
Side Effect Service
Real Time Stream Service
Supports all outgoing integration and webhooks along with

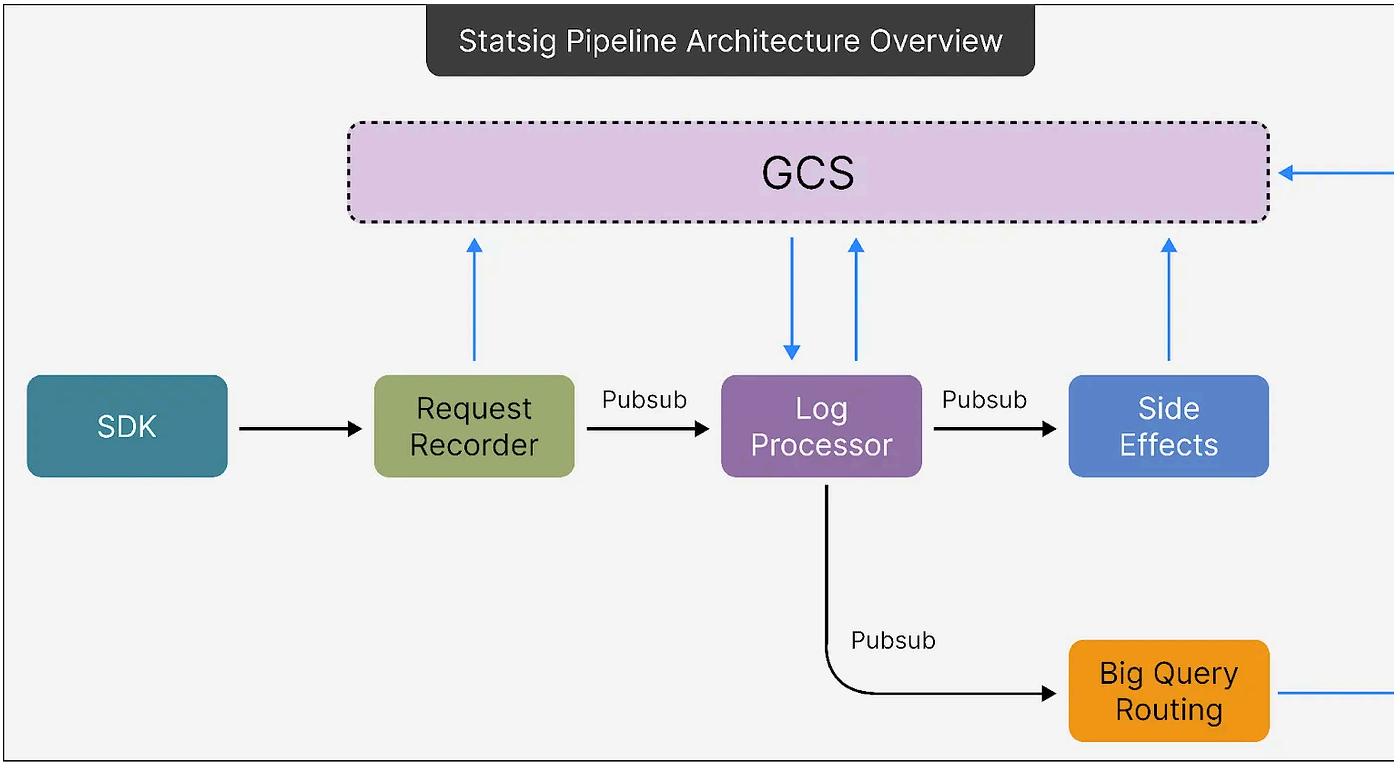


Statsig's Streaming Architecture

On a high level, Statsig's streaming architecture consists of 3 main components:

- **Request Recorder:** This stage of Statsig's pipeline is designed with one main priority: ensuring that no data is ever lost. The top goal here is to make sure tha every piece of data gets captured and stored, no matter what.
- **Log Processing:** In this stage, the raw data collected during request recording g refined and prepared for use. This stage applies specific rules and logic to the r data while also ensuring that the data is accurate.
- **Routing:** This stage acts like a smart traffic controller for data. Once the data h been recorded and processed, this stage ensures it reaches the right destinatior allows the pipeline to handle varied customer requirements without creating separate systems for each use case.

See the diagram below for a quick overview of the architecture:



The Architectural Components

While the high-level view gives a broad look at Statsig's pipeline architecture, let us also look at each component in more detail to gain a better understanding.

1 - Data Ingestion Layer

The data ingestion layer in Statsig's pipeline is the first and one of the most crucial stages of the system.

It is responsible for receiving, authenticating, organizing, and securely storing data a way that prevents loss, even under challenging conditions.

The request recorder is a key functionality of the data ingestion layer, with its speci role being the first step in handling incoming data. However, the data ingestion lay includes the load balancer, authentication, rebatching, and persistence-related functionalities.

See the diagram below:

Here's a simple breakdown of the various steps:

- **From Client SDKs to Load Balancer**:
  - The process starts when data is sent from client SDKs. This data is routed through a load balancer.
  - The load balancer ensures that incoming requests are evenly distributed across servers to avoid overloading any single server. This helps maintain system stability and performance.
- **Service Endpoint & Authorization**:
  - Once the data reaches the server, it is passed to a service endpoint where th system performs authentication. This step ensures that only valid and authorized data batches are processed.
  - After authentication, the data is added to an internal queue. This queue hol the data temporarily until it can be processed further, ensuring no data is lo

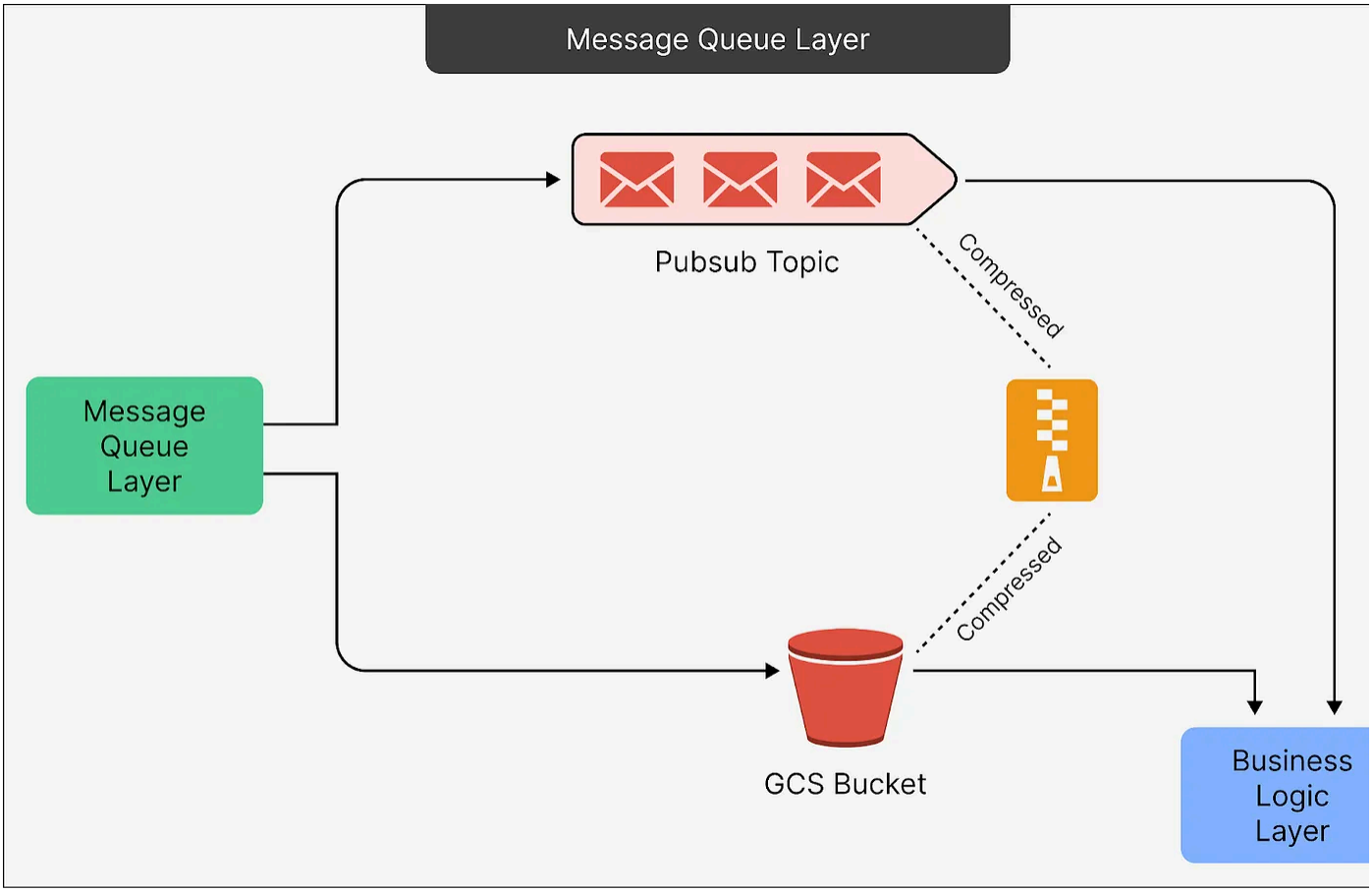
even if the processing system experiences delays.

- **Rebatching for Efficiency**:
  - At this stage, smaller data batches are combined into larger ones, a process called rebatching. Handling fewer, larger batches is more efficient than dealing with many small ones, both in terms of performance and cost.
  - Statsig uses a dynamic configuration system to determine the optimal size these batches.
  - The rebatching process then writes the optimized data batches to Google Cloud Storage (GCS). Using GCS for this purpose is a cost-effective solutio compared to directly streaming smaller batches through high-cost pipeline
- **Persistence for Reliability**:
  - After rebatching, the data is securely stored in a persistence layer, which serves as a semi-permanent storage system. This layer is designed with multiple fallback options, meaning it has backup mechanisms to store data case the primary system encounters issues.
  - The processing units, known as pods, are equipped with sufficient memory handle temporary failures. This ensures the system can continue operating up to an hour during an outage without losing data or affecting performanc
  - These fallback mechanisms extend across different regions and cloud platforms, providing an extra layer of reliability to the system.

2 - Message Queue Layer

The Message Queue Layer is a critical stage in Statsig's pipeline that manages how data flows between different components. This layer is designed to handle enormou volumes of data efficiently while keeping operational costs low.

See the diagram below:



As you can see, it consists of two main components:

The Pub/Sub Topic

Pub/Sub is a serverless messaging system that facilitates communication between different parts of the pipeline. Since it is serverless, there's no need to worry about maintaining servers or managing complex deployments. This reduces overhead for engineering team.

Pub/Sub receives metadata about the data stored in Google Cloud Storage (GCS). Instead of directly storing all the event data, it acts as a pointer system, referring downstream systems to the actual data stored in GCS.

GCS Bucket

Directly using Pub/Sub for all data storage would be prohibitively expensive. Therefore, Statsig offloads most of the data to GCS to reduce storage and operation

costs.

The pipeline writes bulk data into GCS in compressed batches. Pub/Sub stores only the metadata (like file pointers) needed to locate this data in GCS. Downstream components can then use these pointers to retrieve the data when required.

The GCS Bucket stores the actual data in a compressed format, using Zstandard (ZSTD) compression for efficiency. For reference, Zstandard compression is highly efficient, providing better compression rates (around 95%) than other methods like zlib, with lower CPU usage. This ensures data is stored in a smaller footprint while maintaining high processing speeds.

3 - Business Logic Layer

The Business Logic Layer is where the heavy lifting happens in Statsig's pipeline.

This layer is designed to process data while ensuring accuracy and preparing it for final use by various downstream systems. It handles complex logic, customization, a data formatting.

See the diagram below that shows the various steps that happen in this layer:

Let’s look at each step in more detail:

Rebatching

This step combines smaller batches of incoming data into larger ones for processin By handling larger batches, the system reduces the overhead of dealing with multip small data chunks.

The system is designed with an “at least once” guarantee. This means that even if something goes wrong during processing, the data is not lost. It will be retried unti successfully processed.

Stateful Processing To Remove Redundancy

This step focuses on deduplication, which involves filtering out repeated or redund data. For instance, if the same event gets recorded multiple times, this step ensures only one instance is kept.

To achieve this, the system uses caching solutions like Memcached. Memcached provides quick access to previously processed data, enabling the system to identify duplicates efficiently.

Ultimately, deduplication reduces unnecessary processing.

Business Logic Plugins

This layer allows different teams within Statsig to insert custom business logic tailored to their specific needs. For example, one team might add specific tags or attributes to the data, while another might modify event structures for a particular customer.

By using plugins, the system can support diverse use cases without requiring a separate pipeline for each customer. This makes the pipeline both scalable and versatile.

Writer

Once the data has been cleaned, transformed, and customized, the Writer finalizes by writing it to the appropriate destination.

This could be a database, a data warehouse, or an analytics tool, depending on wher the data is needed.

4 - Routing and Integration Layer

The routing and integration layer in Statsig's pipeline is responsible for directing processed data to its final destination.

See the diagram below:

Let’s look at each branch in more detail:

Warehouse Router

The Warehouse Router is responsible for deciding where the data should go based o factors like customer preferences, event types, and priority. It dynamically routes da to various destinations such as BigQuery or other data warehouses.

Here’s how it works:

- For data that does not require immediate access (latency-insensitive), the route uses load jobs. These batch jobs process and upload data in bulk, significantly reducing costs compared to real-time streaming.
- However, for critical, time-sensitive data, the router leverages the storage write API to deliver data in real-time, ensuring low latency but at a higher cost.

The Warehouse Router guarantees efficient resource utilization by distinguishing between latency-sensitive and latency-insensitive data. It saves costs without compromising on performance for urgent tasks.

The Side Effects Service

This service handles external integrations triggered by specific events in the pipeli For example:

- Sending data to third-party systems via webhooks.
- Triggering actions or workflows in external platforms based on predefined rule

It supports any kind of event-level trigger, making it highly customizable for customer-specific workflows.

The Real-Time Event Stream

This service is designed for situations where data needs to be accessed almost instantaneously. For example:

- Real-time dashboards that display user activity.
- Monitoring tools that require immediate updates.

It uses Redis, a fast in-memory data store, to cache and retrieve data in real-time so that customers querying the data experience minimal delays.

The Shadow Pipeline

The Shadow Pipeline is an important testing feature in Statsig's event streaming system.

It acts as a safety net to ensure that any updates or changes to the system don't disr its ability to process over a trillion events a day.

Here's a closer look at how the shadow pipeline works:

- **Parallel Testing Environment**: The shadow pipeline runs alongside the main production pipeline but doesn't affect live data or customer systems. It mirrors production setup, processing the same kinds of data with a new version of the software (release candidate). This allows engineers to test updates in a controll environment without risking disruptions to the main pipeline.
- **Simulating Real Traffic**: To test the shadow pipeline's performance, a couple o million events are sent through it in a large burst. This mimics the high-traffic conditions the system handles daily, providing a realistic scenario for evaluatio
- **Comparing Results**: The outputs of the shadow pipeline are carefully compare with those from the production pipeline. Automated checks look for discrepancies, such as missing or incorrect data, and alert engineers if anythin falls outside expected tolerances.
- **Stress Testing Infrastructure**: The traffic burst also tests the system's ability to handle sudden spikes in data volume. It evaluates features like horizontal pod autoscaling (HPA) that automatically adjusts computing resources to match demand.

Statsig's Cost Optimization Strategies

Statsig employed multiple cost optimization strategies to handle the challenge of processing over a trillion events daily while keeping operational expenses as low as possible.

These strategies involve a mix of technical solutions, infrastructure choices, and design decisions.

Let's break down each key effort in more detail:

GCS Upload via Pub/Sub

Instead of sending all event data directly into Pub/Sub, Statsig writes the majority c the data to Google Cloud Storage (GCS) in a compressed format.

Using GCS is significantly cheaper than relying solely on Pub/Sub for storing large amounts of data. It helps reduce costs while maintaining scalability. Pub/Sub is use only to pass file pointers (metadata) that direct downstream systems to retrieve the data from GCS.

Async Workloads on Spot Nodes

Statsig runs non-time-sensitive tasks (asynchronous workloads) on spot nodes, whi are temporary virtual machines offered at a lower price.

Leveraging spot nodes reduces VM costs without compromising performance for le urgent processes. Also, since these workloads don't require constant uptime, occasional interruptions don't impact the system's overall functionality.

Deduplication with Memcache

A large portion of incoming events may include duplicates, which add unnecessary processing overhead. Deduplication is a key feature that saves processing resources and ensures downstream systems only handle unique data.

To handle deduplication, Statsig uses Memcache to identify and discard duplicate events early in the pipeline.

Zstandard (zstd) Compression

Statsig switched from using zlib compression to ZSTD, a more efficient compressio algorithm.

ZSTD achieves better compression rates (around 95%) while using less CPU power, compared to zlib's 90% compression. This improvement reduced storage requireme

and processing power.

Batching Efficiency via CPU Optimization

Statsig also adjusted the CPU allocation for its request recorder (from 2 CPU to 12 CPU), enabling it to handle larger batches of data more efficiently.

This is because larger batches reduce the number of write operations to storage systems, improving cost efficiency while maintaining high throughput.

Load Jobs vs. Live Streaming

For data that doesn't need to be processed immediately, Statsig uses load jobs to process and upload data in bulk, which is much cheaper.

On the other hand, for time-sensitive data, they use the storage write API, which provides low-latency delivery but at a higher cost.

Differentiating between these two types of data saves money while meeting custom requirements for both real-time and batch processing.

Optimized CPU and Memory Utilization

Statsig tunes CPU and memory usage based on actual host utilization rather than p utilization.

Also, pods are configured without strict usage limits, allowing them to make full us available resources when needed. This prevents underutilization of expensive hardware resources and maximizes cost-effectiveness.

Aggressive Host-Level Resource Stacking

Statsig stacks multiple pods onto a single host aggressively to use every bit of availa CPU and memory.

By fine-tuning flow control and concurrency settings, they prevent resource contention while maintaining high performance. This approach helps achieve cost efficiency at the host level by reducing the number of machines needed.

Conclusion

Statsig's journey to streaming over a trillion events daily shows how a company can achieve massive scale without compromising efficiency through innovative engineering.

By designing a robust data pipeline with key components like a reliable ingestion layer, scalable message queues, and cost-optimized routing and integration layers, Statsig has built an infrastructure capable of supporting rapid growth while maintaining high reliability and performance. Also, leveraging features and tools li Pub/Sub, GCS, and advanced compression techniques, the platform balances the challenges of low latency, data integrity, and cost-effectiveness.

A key differentiator for Statsig is its approach to cost optimization and scalability, achieved through strategies such as using spot nodes, implementing deduplication, and differentiating latency-sensitive from latency-insensitive workloads. These effo not only ensure the system's resilience but also allow them to offer their platform a competitive prices to a wide range of customers.

Reference:

- How Statsig streams 1 trillion events a day

SPONSOR US

Get your product in front of more than 1,000,000 tech professionals.

Our newsletter puts your products and services directly in front of an audience that matters - hundreds of thousands of engineering leaders and senior engineers - who have influence over significant tech decisions and big purchases.

Space Fills Up Fast - Reserve Today

Ad spots typically sell out about 4 weeks in advance. To ensure your ad reaches this influential audience, reserve your space now by emailing **sponsorship@bytebytego.com**.

135 Likes · 10 Restacks

Discussion about this post

Comments Restacks

OO Write a comment...

Lewis Lewis's Substack Dec 19, 2024

Interesting article! Do you happen to know the consistency model of this architecture? Is it ev consistency?

LIKE REPLY

1 reply

1 more comment...